ASHESI UNIVERSITY

My Reflection

~LAB1

Intro to AI Robotics

Throughout this lab, I gained hands-on experience with Test-Driven Development principle (TDD) by utilizing the Red-Green-Refactor cycle; by writing failing unit tests first, I ensured that the system requirements were clearly defined before any actual feature code was implemented. I also learned the core principles of ROS2 architecture, specifically how independent programs called nodes communicate with one another. I implemented 'topics' to handle continuous data streams, allowing a publisher to broadcast messages without needing to know who is subscribing; I built a custom service to handle single request and reply client-server interactions.

Furthermore, I developed a strong foundation in version control using Git and GitHub, mastering how to stage changes, committing them to a local repository, and pushing them to a remote server for safe collaboration.

One of the primary challenges I encountered was correctly configuring the CMake compilation for my custom service interface... Indeed, I learned that ROS2 has strict rules requiring the ament_package() command to be placed at the very end of the CMakeLists.txt file, or the build will fail (I initially placed the command just before the last syntax of the whole code block and it caused an error).I also struggled initially with colcon test returning zero tests; I realized this occurred because the build tool only searches the current directory and downwards, reinforcing the strict rule that colcon commands must always be run from the absolute root of the workspace...

Finally, I faced a Git "diverging branches" issue when attempting to push my code, as the remote repository contained an initial configuration file that my local machine lacked. I

successfully resolved this merge conflict by pulling the remote changes, safely combining the timelines using a terminal text editor, and successfully pushing the synchronized project at the end of it.

Pertaining to some of my observations, I noticed the immense power of ROS2's modularity; for instance, the sensor node simply publishes its simulated distance data to a topic and does not need to know whether a processing node or service node is actually listening! I also observed how system bring-up can be entirely automated using launch files and parameter files. Instead of starting multiple nodes from separate terminals, a single Python launch script was able to dynamically boot the entire system while passing customizable YAML parameters to each node...

Additionally, I recognized the critical importance of keeping version control clean by strictly using a .gitignore file to prevent the massive, auto-generated build, install, and log directories from cluttering the remote repository...

Moving forward, I am curious about how this simulated distance sensor architecture would behave differently if deployed on physical hardware, such as a Raspberry Pi connected to an actual LiDAR or ultrasonic sensor...Furthermore, while we heavily utilized unit tests to verify node initialization, I do wonder how integration tests differ and how we might fully automate them to explicitly test the actual data flowing between the sensor and processing nodes...